

Shutterlike - Full Game Explanation

1. Project and Startup

Your game is a Godot project called Shutterlike. The startup scene is stored in project.godot by a UID pointer, which the engine resolves internally. After startup, the game depends on autoload singletons and scene transitions instead of a single central controller.

The key autoloads are ModLoaderStore, ModLoader, Globals, GlobalSignals, PlayerStats, Weapons, ConsoleCommands, Items, Ezcha, and LimboConsole. These shared objects provide game state, event dispatch, inventory data, and developer tools.

2. Globals.gd - Shared Game State

assets/autoloads/globals.gd stores the player's inventory, currency, progression, item pricing multipliers, and utility methods. It is the main shared state object across the game.

Important fields include bought_weapons (8 weapon slots), bought_items, rupies (currency as BigNumber), round, zone, global_weapon_price_mult, and global_item_price_mult. It also declares a rarity enum and a random seed.

Helper methods in Globals include give_weapon, give_item, swap_weapons, and no_weapons. These manage weapon storage and help the game decide whether the player currently has a weapon.

Globals also provides math and formatting utilities: chance, advanced_chance, get_percentage, round_to_dec, and format. The format method converts large BigNumber values to readable suffixes like T, Qa, Qi, and even Googolplex-scale names.

3. GlobalSignals.gd - Event Bus

assets/autoloads/global_signals.gd defines global signals for game events. It does not hold state; it only provides event dispatch.

Key signals are recalculate_stats, enemy_spawned, enemy_killed, enemy_hit, entity_damaged, round_start, round_end, time_passed, thing_bought, cant_afford, wi_weapon_grabbed, and wi_weapon_dropped. These signals let systems communicate without direct references.

For example, weapons listen for round_start and recalculate_stats; bullets emit enemy_hit; the shop emits thing_bought; enemies react to entity_damaged. This event-driven architecture keeps gameplay systems decoupled.

4. SceneLoader - Smooth Transitions

assets/classes/scene_loader_class.gd performs asynchronous scene loading. It creates a loading screen, starts a threaded resource load, and switches scenes when the new scene is ready.

Stages and the bazar use SceneLoader so transitions are handled cleanly, with progress reporting and an explicit load_finished signal.

5. The Core Gameplay Loop

The game loop is stage → bazar → stage. The player enters a timed stage, fights enemies, and then returns to the bazar shop to purchase weapons and items. After upgrades, the player starts the next round.

Progression is tracked by `Globals.round` and `Globals.zone`. The stage duration and difficulty scale with these values.

6. Stage Logic

`assets/classes/stage_class.gd` defines the generic Stage behavior. It sets a timer based on round and zone, listens for timer timeout, emits `time_passed`, and loads the bazar when the stage finishes.

The formula $\text{time} = \text{ceil}((10 * (\log(\text{Globals.zone}) + 1)) * (\log(\text{Globals.round}) + 1))$ means stage length grows with progression but in a logarithmic way, keeping increases smoother.

`assets/stages/test_stage/test_stage.gd` is the current playable stage. It ensures the player has a weapon by granting a pistol if none are equipped, spawns enemies on timer timeout, and ends the stage when time reaches zero.

7. Enemy Behavior

Enemies are implemented with `assets/classes/enemy_class.gd` as the base. The concrete enemies in `assets/enemies/test_enemy` and `assets/enemies/test_big_enemy` extend that base.

Each enemy targets the player, moves directly toward them, flips its sprite based on movement direction, and dies when health drops below 1. On damage, enemies play a hit animation and spawn a damage label.

Enemy health is scaled by `PlayerStats.damage_multiplier`, so stronger player damage makes enemies harder. Combat is event-driven through `GlobalSignals`.

8. The Bazar Shop

`assets/menus/bazar/bazar.gd` controls the shop screen. It generates four weapon offers and four item offers, connects global signals for purchase handling, and continues to the next stage when the player is ready.

Weapon and item offers are generated using random selection plus rarity-based chance. If a pick fails due to rarity, the shop retries until it gets a valid offer.

9. Weapon Offer Logic

Weapon offers are represented by `assets/menus/bazar/weapon_shop_container.gd`. They show the weapon portrait and formatted price. Buying a weapon stores it in the first empty `Globals.bought_weapons` slot, deducts rupies, emits `thing_bought`, and removes the offer card.

Weapons are not immediately instantiated on purchase; they are stored as data until the next stage begins.

10. Item Offer Logic

Item offers are represented by `assets/menus/bazar/item_shop_container.gd`. They show the item portrait and formatted price. Buying an item appends it to `Globals.bought_items`, deducts rupies, emits `thing_bought`, and removes the card.

Items are stored and later instantiated on the player, or their effects may be applied by item scene logic.

11. Item Data and Effects

`assets/classes/item_data_class.gd` defines item data. Each item includes health, health regeneration, damage multiplier, attack speed, and speed values, along with equation types for each stat: Sum, Sub, Mult, Div, Pow.

When purchased, `bazar.gd` applies each item to `PlayerStats` using the selected equation. This means items can add, subtract, multiply, divide, or exponentiate stats, giving flexible and sometimes powerful effects.

12. Player Stats Display

`assets/menus/bazar/player_stat_container.gd` displays `PlayerStats.stats` in the bazar UI. It formats `BigNumber` values with `Globals.format()` and shows float stats with a multiplier prefix.

This creates a live upgrade screen so the player can see how items change their stats.

13. Player Entity

`assets/classes/player_class.gd` defines the player. It extends `Entity` and inherits the shared `take_damage` behavior. On ready, it initializes health from `PlayerStats`, computes speed and acceleration from `PlayerStats.speed`, creates weapons and items, and emits global `round_start` and `recalculate_stats` signals.

Movement is handled in `_physics_process()`: it reads input axes, smooths velocity with `move_toward`, calls `move_and_slide`, and flips the sprite based on horizontal direction.

Weapon equip is handled in `_create_weapons()`, which loads each weapon scene from `Globals.bought_weapons` into gun position nodes. Item scenes are also instantiated on the player via `_create_items()`.

14. Weapon Data and Runtime Weapons

`assets/classes/weapon_data_class.gd` defines weapon definitions. Each weapon has a name, rarity, price, damage, fire rate, and scene path. The selected weapon data is stored in `Globals` and used to instantiate runtime weapon nodes in stages.

`assets/classes/weapon_class.gd` defines runtime weapon behavior. It computes actual damage and fire rate using `PlayerStats`, listens to combat and round signals, and spawns bullets when it fires.

Weapon damage is calculated as `data.damage * PlayerStats.damage_multiplier`, and fire rate is reduced by attack speed. This links player stat upgrades directly into weapon performance.

15. Specific Weapon Implementations

Pistol fires a single bullet on a timer. AK-47 fires a single bullet and also adds an entity effect. Bazooka fires an explosive bullet. Tactical shotgun fires five bullets in a spread. Each weapon uses `_shoot()` from the base class to create projectile instances.

These weapons are implemented in `assets/weapons/pistol/pistol.gd`, `assets/weapons/ak47/ak_47.gd`, `assets/weapons/bazooka/bazooka.gd`, and `assets/weapons/tactical_shotgun/tactical_shotgun.gd`.

16. Bullet and Projectile Behavior

Bullets are implemented in `assets/classes/bullet_class.gd` as `BulletEntity`. Specific bullet types override `physics_update` and collision reactions.

Default and shotgun bullets move according to `direction * speed` and damage the first enemy they hit. Bazooka bullets explode on impact, spawning an `ExplosionEntity` and dealing damage in a different way.

Each bullet emits `GlobalSignals.enemy_hit` when it strikes an enemy, allowing other systems to react.

17. Damage and Combat Events

Damage is handled by `Entity.take_damage()`. It subtracts health, emits `GlobalSignals.entity_damaged`, and applies any entity effects. Effects are instantiated if they are not already present on the entity.

Enemies listen for `entity_damaged` and display damage text and hit animations. Combat is therefore event-driven and easy to extend.

18. Current Game Experience

In the current state, your game is a timed survival loop with upgrade shop transitions. The player moves with WASD, holds weapons from inventory, and fires automatically via weapon timers and mouse aim. Enemies chase the player and die when their health reaches zero.

After the stage timer expires, the game returns to the bazar. The player can buy weapons and items, then press Continue to start the next round with upgraded stats.

19. Strengths and Design Patterns

The game uses shared singleton state for data, a global signal bus for decoupled communication, data-driven item and weapon definitions, and a reusable scene loader for clean transitions. This makes the game modular and easier to expand.

Weapons, bullets, enemies, and shop offers are separated into data and runtime code, which means new weapons or enemies can be added without changing the core loop.

20. Areas for further extension

The main root launch scene is hidden behind Godot UID references. Enemy AI is currently simple chase behavior. Item effects are applied at purchase time, but item scene logic may provide additional mechanics. The current loop is stage/timer based rather than kill-based. If you want, you can expand with more enemy types, stage objectives, and richer shop mechanics.